

Research Notes on AlexNet

AlexNet

>> Section 1

Subsequent work AlexNet is highly influential, resulting in much subsequent work in using CNNs for computer vision and using GPUs to accelerate deep learning. As of early 2025, the AlexNet paper has been cited over 184,000 times according to Google Scholar. At the time of publication, there was no framework available for GPU-based neural network training and inference. The codebase for AlexNet was released under a BSD license, and had been commonly used in neural network research for several subsequent years. In one direction, subsequent works aimed to train increasingly deep CNNs that achieve increasingly higher performance on ImageNet. In this line of research are GoogLeNet (2014), VGGNet (2014), Highway network (2015), and ResNet (2015). Another direction aimed to reproduce the performance of AlexNet at a lower cost. In this line of research are SqueezeNet (2016), MobileNet (2017), EfficientNet (2019). Geoffrey Hinton, Ilya Sutskever, and Alex Krizhevsky formed DNNResearch soon afterwards and sold the company, and the AlexNet source code along with it, to Google. There had been improvements and reimplementations for the AlexNet, but the original version as of 2012, at the time of its winning of ImageNet, had been released under BSD-2 license via Computer History Museum.

>> Section 2

Training The ImageNet training set contained 1.2 million images. The model was trained for 90 epochs over a period of five to six days using two Nvidia GTX 580 GPUs (3GB each). These GPUs have a theoretical performance of 1.581 TFLOPS in float32 and were priced at US\$500 upon release. Each forward pass of AlexNet required approximately 1.43 GFLOPs. Based on these values, the two GPUs together were theoretically capable of performing over 2,200 forward passes per second under ideal conditions. The dataset images were stored in JPEG format. They took up 27GB of disk. The neural network took up 2GB of RAM on each GPU, and around 5GB of system RAM during training. The GPUs were responsible for training, while the CPUs were responsible for loading images from disk, and data-augmenting the images. AlexNet was trained with momentum gradient descent with a batch size of 128 examples, momentum of 0.9, and weight decay of 0.0005. Learning rate started at 10⁻² and was manually decreased 10-fold whenever validation error appeared to stop decreasing. It was reduced three times during training, ending at 10⁻⁵. It used two forms of data augmentation, both computed on the fly on the CPU, thus "computationally free":

>> Section 3

Each image from ImageNet was first scaled, so that its shorter side was of length 256. Then the central 256×256 patch was cropped out and normalized (dividing the pixel values so that they fall between 0 and 1, then subtracting by [0.485, 0.456, 0.406], then dividing by [0.229, 0.224, 0.225]. These are the mean and standard deviations for ImageNet, so this whitens the input data). Extracting random 224×224 patches (and their horizontal reflections) from the 256×256 crop. This increases the size of the training set 2048-fold. Randomly shifting the RGB value of each image along the three principal directions of the RGB values of its pixels. The resolution 224×224 was picked, because $256 - 16 - 16 = 224$, meaning that given a 256×256 image, framing out a width of 16 on its 4 sides results in a 224×224 image. It used local response normalization, and dropout regularization with drop probability 0.5. All weights were initialized as gaussians with 0 mean and 0.01 standard deviation. Biases in convolutional layers 2, 4, 5, and all fully-connected layers, were initialized to constant 1 to avoid the dying ReLU problem. At test time, to use a trained AlexNet for predicting the class of an image, that image is first scaled, so that its shorter side was of length 256. Then the central 256×256 patch was cropped out. Then, the five 224 × 224 patches (the four corner patches and the center patch) as well as their horizontal reflections are computed, 10 patches in all. The network's predicted probabilities on all 10 patches are averaged, and that is the final predicted probability.

>> Section 4

ImageNet competition The version they used to enter the 2012 ImageNet competition was an ensemble of 7 AlexNets. Specifically, they trained 5 AlexNets of the previously described architecture (with 5 CONV layers) on the ILSVRC-2012 training set (1.2 million images). They also trained 2 variant AlexNets, obtained by adding one extra CONV layer over the last pooling layer. These were trained by first training on the entire ImageNet Fall 2011 release (15 million images in 22K categories), and then finetuning it on the ILSVRC-2012 training set. The final system of 7 AlexNets was used by averaging their predicted probabilities.

>> Section 5

AlexNet is a convolutional neural network architecture developed for image classification tasks, notably achieving prominence through its performance in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC). It classifies images into 1,000 distinct object categories and is regarded as the first widely recognized application of deep convolutional networks in large-scale visual recognition. Developed in 2012 by Alex Krizhevsky in

Research Notes on AlexNet

AlexNet

collaboration with Ilya Sutskever and his Ph.D. advisor Geoffrey Hinton at the University of Toronto, the model contains 60 million parameters and 650,000 neurons. The original paper's primary result was that the depth of the model was essential for its high performance, which was computationally expensive, but made feasible due to the utilization of graphics processing units (GPUs) during training. The three formed team SuperVision and submitted AlexNet in the ImageNet Large Scale Visual Recognition Challenge on September 30, 2012. The network achieved a top-5 error rate of 15.3% to win the contest, more than 10.8% above the runner-up. The architecture influenced a large number of subsequent work in deep learning, especially in applying neural networks to computer vision.

>> Section 6

Architecture AlexNet contains eight layers: the first five are convolutional layers, some of them followed by max-pooling layers, and the last three are fully connected layers. The network, except the last layer, is split into two copies, each run on one GPU, because the network did not fit the VRAM of a single Nvidia GTX 580 3GB GPU. The entire structure can be written as $(\text{CONV} \rightarrow \text{RN} \rightarrow \text{MP})_2 \rightarrow (\text{CONV}_3 \rightarrow \text{MP}) \rightarrow (\text{FC} \rightarrow \text{DO})_2 \rightarrow \text{Linear} \rightarrow \text{softmax}$ where CONV = convolutional layer (with ReLU activation) RN = local response normalization MP = max-pooling FC = fully connected layer (with ReLU activation) Linear = fully connected layer (without activation) DO = dropout Notably, the convolutional layers 3, 4, 5 were connected to one another without any pooling or normalization. It used the non-saturating ReLU activation function, which trained better than tanh and sigmoid.